

Práctica Nro. 7

Sistemas y Tipos de Datos

Objetivo: Comprender las nociones fundamentales sobre las diversas propiedades de los sistemas de tipos y los tipos de datos.

Ejercicio 1: Sistema de tipos

1. ¿Qué es un sistema de tipos y cuál es su principal función?

Un sistema de tipos es el conjunto de reglas que utiliza un lenguaje de programación para estructurar y organizar sus tipos de datos. Define cómo se crean, relacionan y utilizan los tipos dentro del lenguaje.

Su función principal es escribir programas seguros, evitando errores relacionados con el uso incorrecto de los tipos de datos.

Además, un sistema de tipos: permite expresar tipos predefinidos o crear nuevos tipos, define reglas de compatibilidad, equivalencia e inferencia de tipos y ayuda a comprender mejor la semántica del lenguaje.

2. Definir y contrastar las definiciones de un sistema de tipos fuerte y débil (probablemente en la bibliografía se encuentren dos definiciones posibles. Volcar ambas en la respuesta). Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar.

Primera definición (general): un sistema de tipos es fuerte cuando especifica restricciones sobre cómo pueden operar entre sí valores de distintos tipos. Un sistema débil impone menos restricciones o permite más conversiones automáticas.

Segunda definición (Type Safety): un lenguaje es **fuertemente tipado** si su sistema de tipos impone restricciones que aseguran que no se producirán errores de tipo durante la ejecución. Es decir, todos los errores de tipo son detectados por el sistema de tipos. Básicamente, la diferencia principal es que el **tipado fuerte** prioriza la seguridad y evita errores de tipo, mientras que **tipado débil** ofrece mayor flexibilidad, pero aumenta la posibilidad de errores durante la ejecución.

- Lenguajes fuertemente tipados:
 - Python: es fuertemente tipado porque no permite operaciones entre tipos incompatibles sin conversión explícita. Por ejemplo: `"5" + 3` produce un error de tipo.

- Gobstones: lenguaje fuertemente tipado y de tipado estático.
- Lenguajes débilmente tipados:
 - C: el material lo clasifica como débilmente tipado debido a que permite numerosas conversiones implícitas y operaciones que pueden violar la seguridad de tipos.
 - C++: hereda muchas de las conversiones implícitas y mecanismos de casting de C, permitiendo operaciones menos controladas sobre los tipos.
- 3. Además de la clasificación anterior, también es posible caracterizar el tipado como estático o dinámico. ¿Qué significa esto? Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar.

Esta clasificación depende del momento en que se verifica el tipo.

Tipado estático: las verificaciones de tipos se realizan durante la compilación. Las variables tienen un tipo determinado antes de ejecutar el programa. Las principales características son, que los errores de tipos suelen detectarse antes de ejecutar el programa, y generalmente requiere declaraciones de tipos o mecanismos de inferencia. Por ejemplo C y Java tienen tipado estático.

Ejemplo en C:

```
int x = 10;
x = "hola"; // Error de compilación
```

Tipado dinámico: las verificaciones de tipos se realizan durante la ejecución. El tipo se asocia al valor y no necesariamente a la variable. Como característica tiene mayor flexibilidad, más comprobaciones en tiempo de ejecución y los errores de tipos pueden aparecer mientras el programa se ejecuta. Python y JavaScript tienen tipado dinámico.

Ejemplo en Python:

```
x = 10
x = "hola"
```

La variable puede almacenar valores de distintos tipos.

Importante: las clasificaciones son independientes. Por ejemplo, Python es fuertemente tipado y dinámico, mientras que C es débilmente tipado y estático.

Ejercicio 2: Tipos de datos

1. Dar una definición de tipo de dato.

Un tipo de dato es una clasificación que determina el conjunto de valores que puede tomar una variable y las operaciones que pueden realizarse sobre esos valores. Los tipos de datos permiten organizar la información y proporcionan seguridad al programa mediante el sistema de tipos.

2. ¿Qué es un tipo predefinido elemental? Dar ejemplos.

Un **tipo predefinido elemental** es un tipo de dato incorporado por defecto en el lenguaje de programación. Sus valores y operaciones ya están definidos por el propio lenguaje y no necesitan ser creados por el programador.

Ejemplos:

- Integer (entero): números enteros → 25
- Real / Float: números con decimales → 3.14
- Boolean: valores lógicos → true
- Character (char): caracteres individuales → 'A'
- String: cadenas de caracteres (algunos lenguajes lo consideran básico). → "hola"

3. ¿Qué es un tipo definido por el usuario? Dar ejemplos.

Un **tipo definido por el usuario** es un nuevo tipo de dato creado por el programador a partir de otros tipos existentes. Este proceso se conoce como abstracción de datos. Los nuevos tipos definidos por el usuario se denominan Tipos Abstractos de Datos (TAD).

Por ejemplo: una clase Persona definida con lenguaje Java.

Los tipos definidos por el usuario permiten representar entidades del problema de forma más clara y favorecen el encapsulamiento y la abstracción.

Ejercicio 3: Tipos compuestos

1. Dar una breve definición de: producto cartesiano (en la bibliografía puede aparecer también como product type), correspondencia finita, uniones (en la bibliografía puede aparecer también como sum type) y tipos recursivos.

Producto Cartesiano (Product Type): un producto cartesiano es un tipo compuesto formado por la combinación de varios tipos de datos. Cada valor del nuevo tipo está compuesto por un valor de cada uno de los tipos que lo integran.

Por ejemplo:

Persona = (Nombre, Edad)

Si:

Nombre = {"Ana", "Juan"}

Edad = {20, 21}

Entonces algunos valores posibles son:

(Ana,20)

(Ana,21)

(Juan,20)

(Juan,21)

Las estructuras (struct), registros (record) o clases son ejemplos de producto cartesiano.

Correspondencia Finita: es una estructura que asocia elementos de un conjunto con elementos de otro conjunto mediante una cantidad finita de pares clave-valor. Se utiliza para representar diccionarios, tablas o mapas. Por ejemplo:

{"Juan" → 8,

"Ana" → 10,

"Pedro" → 6}

Aquí cada nombre se corresponde con una nota.

Uniones (Sum Type): una unión permite que una variable pueda contener un valor perteneciente a uno entre varios tipos posibles, pero solamente uno de ellos a la vez.

Por ejemplo:

Resultado =

Exito(String) | Error(Integer)

Un valor puede ser:

Exito("Operación realizada")

Error(404)

pero nunca ambos simultáneamente.

En C existe algo similar mediante:

union Dato {

int numero;

float decimal;

};

Tipos Recursivos: es aquel que se define utilizando una referencia a sí mismo. Se emplean para representar estructuras dinámicas como listas, árboles o grafos. Ejemplo de lista enlazada:

Lista =

Vacia | Nodo(valor, Lista)

Los tipos recursivos permiten representar estructuras de tamaño variable y potencialmente ilimitado.

2. Identificar a qué clase de tipo de datos pertenecen los siguientes extractos de código. En algunos casos puede corresponder más de una:

Java <pre>class Persona { String nombre; String apellido; int edad; }</pre>	C <pre>typedef struct _nodoLista { void *dato; struct _nodoLista *siguiente } nodoLista; typedef struct _lista { int cantidad; nodoLista *primero } Lista;</pre>	C <pre>union codigo { int numero; char id; };</pre>
Ruby <pre>hash = { uno: 1, dos: 2, tres: 3, cuatro: 4 }</pre>	PHP <pre>function doble(\$x) { return 2 * \$x; }</pre>	Python <pre>tuple = ('physics', 'chemistry', 1997, 2000)</pre>
Haskell <pre>data ArbolBinarioInt = Nil Nodo int (ArbolBinarioInt dato) (ArbolBinarioInt dato)</pre> Ayuda para interpretar: 'ArbolBinarioInt' es un tipo de dato que puede ser Nil ("vacío") o un Nodo con un dato número entero (int) junto a un árbol como hijo izquierdo y otro árbol como hijo derecho	Haskell <pre>data Color = Rojo Verde Azul</pre> Ayuda para interpretar: 'Color' es un tipo de dato que puede ser Rojo, Verde o Azul.	

- 1) Java → Class Persona es de tipo Producto Cartesiano.
- 2) C → typedef struct _nodoLista es de tipo Recursivo.
- 3) C → unión código es de tipo Producto Cartesiano.
- 4) Ruby → hash es de tipo Correspondencia Finita.
- 5) PHP → function doble (\$x) es de tipo Correspondencia Finita.
- 6) Python → tuple es de tipo Producto Cartesiano.
- 7) Haskell → data ArbolBinarioInt es de tipo Recursivo y Unión (o es un nodo con hijos [recursión], o es nil pero no ambos a la vez [unión]).
- 8) Haskell → data Color es de tipo unión (puede ser una de todas las opciones dadas, pero no todas a la vez).

Ejercicio 4: Mutabilidad/Inmutabilidad

- Definir mutabilidad e inmutabilidad respecto a un dato. Dar ejemplos en al menos 2 lenguajes. TIP: indagar sobre los tipos de datos que ofrece Python y sobre la operación #freeze en los objetos de Ruby.

Un dato es mutable cuando su estado o contenido puede modificarse después de haber sido creado. En cambio, un dato es inmutable cuando, una vez creado, no puede alterarse; cualquier "cambio" produce en realidad un nuevo objeto.

Ejemplo en Python: cadenas, enteros y tuplas son inmutables mientras que listas, conjuntos y diccionarios son mutables pues los puedo cambiar luego de ser creados.

- ➔ *lista = [1, 2, 3] //ejemplo objeto mutable, modifico el tamaño de la lista*
lista.append(4)
print(lista) # [1, 2, 3, 4]
- ➔ *cadena = "hola" //las cadenas son inmutables, en este caso se crea nuevo objeto*
cadena = cadena + " mundo"
print(cadena) # "hola mundo"

Ejemplo en Ruby: la mayoría de los objetos son mutables por defecto, pero puedo hacerlos inmutables mediante freeze

- ➔ *nombre = "Juan" //Ruby permite convertir objeto a inmutable mediante freeze*
nombre.freeze
nombre << " Pérez"
RuntimeError: can't modify frozen String

- Dado el siguiente código:

```
a = Dato.new(1)  
a = Dato.new(2)
```

¿Se puede afirmar entonces que el objeto "Dato.new(1)" es mutable? Justificar la respuesta sea por afirmativa o por la negativa.

No puedo asegurar nada porque lo único que muestra el código es una reasignación de variables y esto no implica mutabilidad pues no altera el estado interno del objeto.

Ejercicio 5: Manejo de punteros

- ¿Permite C tomar el L-valor de las variables? Ejemplificar.

Sí, C permite obtener la dirección de memoria (L-valor) de una variable mediante el operador &. Así, se puede modificar el contenido de esa ubicación mediante asignaciones u otras operaciones. Ejemplo:

```
int x = 10; //la variable x contiene el valor 10
```

```
int *p; //p almacena la dirección de memoria
```

```
p = &x; //&x representa el L-valor en donde está almacenado x
```

- ¿Qué problemas existen en el manejo de punteros? Ejemplificar.

Violación de tipos: ocurre cuando un puntero se usa para acceder a datos de un tipo diferente al esperado, lo que puede llevar a comportamientos indefinidos o errores.

Referencias sueltas (dangling): puntero que contiene una dirección de una variable dinámica que fue desalocada, entonces si uso el puntero me va a tirar error.

Punteros no inicializados: contiene una dirección aleatoria, lo cual puede controlar accesos descontrolados a memoria, requiere verificación dinámica.

Punteros y uniones discriminadas: puede permitir accesos indebidos si no se verifica correctamente el tipo de datos almacenado, accediendo a campos incorrectos.

Alias: si dos o más punteros apuntan a la misma dirección de memoria la modificación de uno afecta a los demás, esto puede generar resultados inesperados.

Liberación de memoria – Objetos perdidos: si los objetos en la heap dejan de ser accesibles (porque ninguna variable los apunta) se convierten en basura y la memoria no los libera, pero siguen ocupando memoria.

Ejercicio 6: TAD

- ¿Qué características debe cumplir una unidad para que sea un TAD?

Un Tipo Abstracto de Datos (TAD) combina una representación de datos con las operaciones que pueden aplicarse sobre ellos. Para que una unidad implemente un TAD debe cumplir principalmente:

Encapsulamiento: la representación y las operaciones se describen en una única unidad sintáctica. Refleja las abstracciones descubiertas en el diseño.

Ocultamiento de información: la implementación interna y la representación de los datos permanecen ocultas al usuario. Refleja los niveles de abstracción.

Abstracción: el usuario utiliza las operaciones definidas sin necesidad de conocer cómo se implementan internamente.

En resumen: un TAD debe ofrecer una interfaz pública de operaciones y ocultar los detalles de implementación, garantizando encapsulamiento y ocultamiento de información.

- Dar algunos ejemplos de TAD en lenguajes tales como ADA, Java, Python, entre otros.

Algunos ejemplos de TAD por ejemplo en Java o Python serían las clases, o en ADA podrían ser los Package.